

Dependency injection

In software engineering, **dependency injection** is a programming technique in which an object or function receives other objects or functions that it requires, as opposed to creating them internally. Dependency injection aims to separate the concerns of constructing objects and using them, leading to loosely coupled programs.^{[1][2][3]} The pattern ensures that an object or function that wants to use a given service should not have to know how to construct those services. Instead, the receiving "client" (object or function) is provided with its dependencies by external code (an "injector"), which it is not aware of.^[4] Dependency injection makes implicit dependencies explicit and helps solve the following problems:^[5]

- How can a class be independent from the creation of the objects it depends on?
- How can an application and the objects it uses support different configurations?

Dependency injection is often used to keep code in-line with the dependency inversion principle.^{[6][7]}

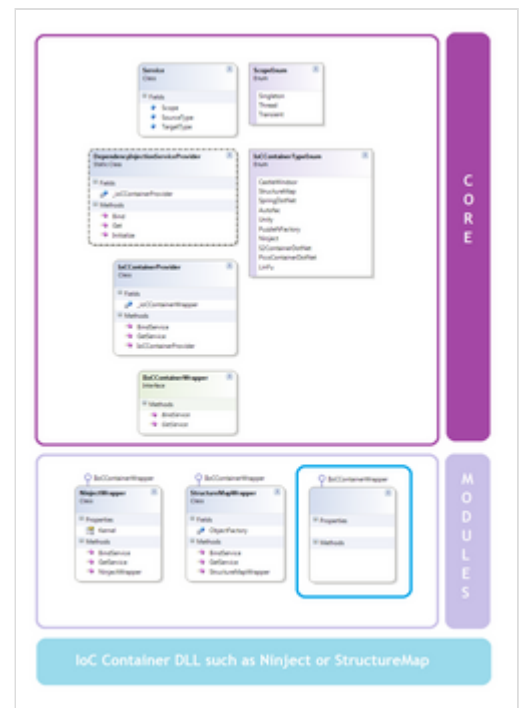
In statically typed languages using dependency injection means that a client only needs to declare the interfaces of the services it uses, rather than their concrete implementations, making it easier to change which services are used at runtime without recompiling.

Application frameworks often combine dependency injection with inversion of control. Under inversion of control, the framework first constructs an object (such as a controller), and then passes control flow to it. With dependency injection, the framework also instantiates the dependencies declared by the application object (often in the constructor method's parameters), and passes the dependencies into the object.^[8]

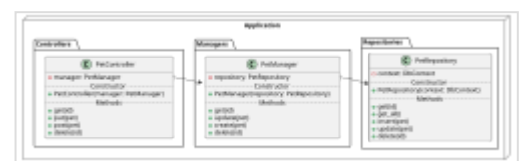
Dependency injection implements the idea of "inverting control over the implementations of dependencies", which is why certain Java frameworks generically name the concept "inversion of control" (not to be confused with inversion of control flow).^[9]

Roles

Dependency injection involves four roles: services, clients, interfaces, and injectors.



Dependency injection is often used alongside specialized frameworks, known as 'containers', to facilitate program composition.



PetManager gets injected into PetController and PetRepository gets injected into PetManager

Services and clients

A service is any class which contains useful functionality. In turn, a client is any class which uses services. The services that a client requires are the client's *dependencies*.

Any object can be a service or a client; the names relate only to the role the objects play in an injection. The same object may even be both a client (it uses injected services) and a service (it is injected into other objects). Upon injection, the service is made part of the client's state, available for use.^[12]

Interfaces

Clients should not know how their dependencies are implemented, only their names and API. A service which retrieves emails, for instance, may use the IMAP or POP3 protocols behind the scenes, but this detail is likely irrelevant to calling code that merely wants an email retrieved. By ignoring implementation details, clients do not need to change when their dependencies do.

Injectors

The **injector**, sometimes also called an assembler, container, provider or factory, introduces services to the client.

The role of injectors is to construct and connect complex object graphs, where objects may be both clients and services. The injector itself may be many objects working together, but must not be the client, as this would create a circular dependency.

Because dependency injection separates how objects are constructed from how they are used, it often diminishes the importance of the **new** keyword found in most object-oriented languages. Because the framework handles creating services, the programmer tends to only directly construct value objects which represents entities in the program's domain (such as an **Employee** object in a business app or an **Order** object in a shopping app).^{[13][14][15][16]}

Analogy

As an analogy, cars can be thought of as services which perform the useful work of transporting people from one place to another. Car engines can require gas, diesel or electricity, but this detail is unimportant to the client—a passenger—who only cares if it can get them to their destination.

Cars present a uniform interface through their pedals, steering wheels and other controls. As such, which engine they were 'injected' with on the factory line ceases to matter and drivers can switch between any kind of car as needed.

Dependency injection for five-year-olds

When you go and get things out of the refrigerator for yourself, you can cause problems. You might leave the door open, you might get something Mommy or Daddy don't want you to have. You might even be looking for something we don't even have or which has expired.

What you should be doing is stating a need, "I need something to drink with lunch," and then we will make sure you have something when you sit down to eat something.

—John Munsch, 28 October 2009.^{[2][10][11]}

Advantages and disadvantages

Advantages

A basic benefit of dependency injection is decreased coupling between classes and their dependencies.^{[17][18]}

By removing a client's knowledge of how its dependencies are implemented, programs become more reusable, testable and maintainable.^[19]

This also results in increased flexibility: a client may act on anything that supports the intrinsic interface the client expects.^[20]

More generally, dependency injection reduces boilerplate code, since all dependency creation is handled by a singular component.^[19]

Finally, dependency injection allows concurrent development. Two developers can independently develop classes that use each other, while only needing to know the interface the classes will communicate through. Plugins are often developed by third-parties that never even talk to developers of the original product.^[21]

Testing

Many of dependency injection's benefits are particularly relevant to unit-testing.

For example, dependency injection can be used to externalize a system's configuration details into configuration files, allowing the system to be reconfigured without recompilation. Separate configurations can be written for different situations that require different implementations of components.^[22]

Similarly, because dependency injection does not require any change in code behavior, it can be applied to legacy code as a refactoring. This makes clients more independent and are easier to unit test in isolation, using stubs or mock objects, that simulate other objects not under test.

This ease of testing is often the first benefit noticed when using dependency injection.^[23]

Disadvantages

Critics of dependency injection argue that it:

- Creates clients that demand configuration details, which can be onerous when obvious defaults are available.^[21]
- Makes code difficult to trace because it separates behavior from construction.^[21]
- Is typically implemented with reflection or dynamic programming, hindering IDE automation.^[24]
- Typically requires more upfront development effort.^[25]
- Encourages dependence on a framework.^{[26][27][28]}

Types of dependency injection

There are several ways in which a client can receive injected services:^[29]

- Constructor injection, where dependencies are provided through a client's class constructor.
- Method Injection, where dependencies are provided to a method only when required for specific functionality.
- Setter injection, where the client exposes a setter method which accepts the dependency.
- Interface injection, where the dependency's interface provides an injector method that will inject the dependency into any client passed to it.

In some frameworks, clients do not need to actively accept dependency injection at all. In Java, for example, reflection can make private attributes public when testing and inject services directly.^[30]

Without dependency injection

In the following Java example, the `Client` class contains a `Service` member variable initialized in the constructor. The client directly constructs and controls which service it uses, creating a hard-coded dependency.

```
public class Client {
    private Service service;

    Client() {
        // The dependency is hard-coded.
        this.service = new ExampleService();
    }
}
```

Constructor injection

The most common form of dependency injection is for a class to request its dependencies through its constructor. This ensures the client is always in a valid state, since it cannot be instantiated without its necessary dependencies.

```
public class Client {
    private Service service;

    // The dependency is injected through a constructor.
    Client(Service service) {
        if (service == null) {
            throw new IllegalArgumentException("service must not be null");
        }
        this.service = service;
    }
}
```

Avoiding singletons

Dependency injection can be used to avoid the singleton pattern. The singleton pattern enforces a structural restriction of only one instance of a class, but dependency injection focuses on how dependencies are passed around, avoiding the necessity of objects to look up their own global state.^[31]

For example, in the following example, the `OrderService` is tightly coupled to `DatabaseConnection`; the database cannot be swapped and a different instance cannot be used.

```
// a concrete singleton class
class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {
        // database resource initialization
    }

    public static synchronized DatabaseConnection instance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void executeQuery(String sql) {
        System.out.printf("Executing: %s\n");
    }
}

// a tightly-coupled consumer class
class OrderService {
    public void placeOrder(String orderId) {
        // hardcoded dependency lookup makes this impossible to unit test
        DatabaseConnection db = DatabaseConnection.instance();
        db.executeQuery(String.format("INSERT INTO orders VALUES ('%s')"));
    }
}
```

Dependency injection avoids asking how the database is created, or how many instances exist, and only asks for the Database interface in its constructor:

```
// an abstract interface
interface Database {
    void executeQuery(String sql);
}

// implement the concrete class (no singleton logic)
class SqlDatabase implements Database {
    @Override
    public void executeQuery(String sql) {
        System.out.println("Executing SQL: %s", sql);
    }
}

// the consumer class accepts dependency via constructor injection
class OrderService {
    private final Database db;

    // The dependency is injected from the outside
    public OrderService(Database db) {
        this.db = db;
    }

    public void placeOrder(String orderId) {
        db.executeQuery(String.format("INSERT INTO orders VALUES ('%s')", orderId));
    }
}
```

Method injection

Dependencies are passed as arguments to a specific method, allowing them to be used only during that method's execution without maintaining a long-term reference. This approach is particularly useful for temporary dependencies or when different implementations are needed for various method calls.

```
public class Client {
    public void performAction(Service service) {
        if (service == null) {
            throw new IllegalArgumentException("service must not be null");
        }
        service.execute();
    }
}
```

Setter injection

By accepting dependencies through a setter method, rather than a constructor, clients can allow injectors to manipulate their dependencies at any time. This offers flexibility, but makes it difficult to ensure that all dependencies are injected and valid before the client is used.

```
public class Client {
    private Service service;

    // The dependency is injected through a setter method.
    public void setService(Service service) {
        if (service == null) {
            throw new IllegalArgumentException("service must not be null");
        }
        this.service = service;
    }
}
```

Interface injection

With interface injection, dependencies are completely ignorant of their clients, yet still send and receive references to new clients.

In this way, the dependencies become injectors. The key is that the injecting method is provided through an interface.

An assembler is still needed to introduce the client and its dependencies. The assembler takes a reference to the client, casts it to the setter interface that sets that dependency, and passes it to that dependency object which in turn passes a reference to itself back to the client.

For interface injection to have value, the dependency must do something in addition to simply passing back a reference to itself. This could be acting as a factory or sub-assembler to resolve other dependencies, thus abstracting some details from the main assembler. It could be reference-counting so that the dependency knows how many clients are using it. If the dependency maintains a collection of clients, it could later inject them all with a different instance of itself.

```
package org.wikipedia.examples;

import java.util.HashSet;
import java.util.Set;
```

```

interface ServiceSetter {
    void setService(Service service);
}

class Client implements ServiceSetter {
    private Service service;

    @Override
    public void setService(Service service) {
        if (service == null) {
            throw new IllegalArgumentException("service must not be null");
        }
        this.service = service;
    }
}

class ServiceInjector {
    private final Set<ServiceSetter> clients = new HashSet<>();

    public void inject(ServiceSetter client) {
        this.clients.add(client);
        client.setService(new ExampleService());
    }

    public void switch() {
        for (Client client : this.clients) {
            client.setService(new AnotherExampleService());
        }
    }
}

class ExampleService implements Service {}

class AnotherExampleService implements Service {}

```

Assembly

The simplest way of implementing dependency injection is to manually arrange services and clients, typically done at the program's root, where execution begins.

```

public class Program {
    public static void main(String[] args) {
        // Build the service.
        Service service = new ExampleService();

        // Inject the service into the client.
        Client client = new Client(service);

        // Use the objects.
        System.out.println(client.greet());
    }
}

```

Manual construction may be more complex and involve builders, factories, or other construction patterns.

Frameworks

Manual dependency injection is often tedious and error-prone for larger projects, promoting the use of frameworks which automate the process. Manual dependency injection becomes a dependency injection framework once the constructing code is no longer custom to the application and is instead universal.^[32] While useful, these tools are not required in order to perform dependency injection.^{[33][34]}


```

/* greeter.service.ts */
import { Injectable, inject } from '@angular/core';
import { DOCUMENT } from '@angular/common';

export interface IGreeterService {
  greet(text: string): void;
}

@Injectable({
  providedIn: 'root'
})
export class GreeterService implements IGreeterService {
  // Use inject(DOCUMENT) and access defaultView to get the Window object
  private window: Window = inject(DOCUMENT).defaultView as Window;

  greet(text: string): void {
    this.window.alert(text);
  }
}

```

```

<!-- my-controller.component.html -->
<div>
  <button (click)="sayHello()">Hello</button>
</div>

```

```

/* my-controller.component.ts */
import { GreeterService } from '../greeter.service.ts';
import { Component, inject } from '@angular/core';
@Component({
  selector: 'app-my-controller',
  template: 'my-controller.component.html'
})
export class MyControllerComponent {

  private greeter = inject(GreeterService);

  sayHello(): void {
    this.greeter.greet('Hello World');
  }
}

```

The `inject` (<https://angular.dev/api/core/inject>) function triggers the injector to create an instance of the `GreeterService` and its dependencies (in this case, a `Document` (<https://developer.mozilla.org/fr/docs/Web/API/Document>) instance to access the window object).

C++

This sample provides an example of constructor injection.

```

import std;

class DatabaseConnection {
public:
  void connect() {
    std::println("Connecting to database...");
  }
};

class DatabaseService {
private:
  DatabaseConnection& dbConn;
public:
  explicit DatabaseService(DatabaseConnection& db):
    dbConn{db} {}
}

```

```

    void execute() {
        dbConn.connect();
        std::println("Executing database service...");
    }
};

int main(int argc, char* argv[]) {
    DatabaseConnection db;
    DatabaseService sv(db);

    sv.execute();
}

```

This sample provides an example of interface injection.

```

import std;

using std::expected;
using std::shared_ptr;
using std::unexpected;

class IConnection {
public:
    enum class Error {
        NO_CONNECTION,
        // more errors here
    };

    virtual void connect() = 0;
    virtual ~IConnection() = default;
};

class DatabaseConnection: public IConnection {
public:
    DatabaseConnection() = default;

    void connect() override {
        std::println("Connecting to database...");
    }
};

class DatabaseService {
private:
    shared_ptr<IConnection> conn;
public:
    DatabaseService() = default;

    void setConnection(shared_ptr<IConnection> nextConn) noexcept {
        conn = nextConn;
    }

    expected<void, IConnection::Error> execute() {
        if (conn) {
            conn->connect();
            std::println("Executing database service...");
        } else {
            return unexpected(IConnection::Error::NO_CONNECTION);
        }
    }
};

int main(int argc, char* argv[]) {
    shared_ptr<DatabaseConnection> db = std::make_shared<DatabaseConnection>();
    DatabaseService sv;
    sv.setConnection(db);
    sv.execute();
}

```

Historically, because C++ lacked reflection, libraries could not "see" constructor signatures or class member types, forcing the usage of manual dependency injection or metaprogramming libraries such as Boost.DI to manually map arguments to dependency injections.^[40] This is simplified greatly using C++26's function parameter reflection and annotations.^{[41][42]}

```
import std;

using std::string;

class IDatabase { /* ... */ };
class FastDatabase : public IDatabase { /* ... */ };
class BasicDatabase : public IDatabase { /* ... */ };

class Foo {
private:
    // ...
public:
    [[Inject]]
    Foo(const IDatabase& primary, const IDatabase& secondary, const string& s, int i) {
        // ...
    }
};

class Bar {
private:
    // ...
public:
    [[Inject]]
    Bar(const IDatabase& primary, int i) {
        // ...
    }
};

Injector<IDatabase> injector = Injector<IDatabase>::Builder()
    .addService("primary", std::make_unique<FastDatabase>())
    .addService("secondary", std::make_unique<BasicDatabase>())
    .build();

Foo foo = injector.build<Foo>("abc", 1);
Bar bar = injector.build<Bar>(2);
```

C#

This sample provides an example of constructor injection in C#.

```
namespace Wikipedia.Examples;

using System;

// Our client will only know about this interface, not which specific gamepad it is using.
interface IGamePadFunctionality
{
    string GetGamePadName();
    void SetVibrationPower(float power);
}

// The following services provide concrete implementations of the above interface.

class XboxGamePad : IGamePadFunctionality
{
    float vibrationPower = 1.0f;

    public string GetGamePadName() => "Xbox controller";

    public void SetVibrationPower(float power) => this.vibrationPower = Math.Clamp(power, 0.0f, 1.0f);
}
```

```

class PlayStationJoystick : IGamePadFunctionality
{
    float vibratingPower = 100.0f;

    public string GetGamePadName() => "PlayStation controller";

    public void SetVibrationPower(float power) => this.vibratingPower = Math.Clamp(power * 100.0f,
0.0f, 100.0f);
}

class SteamController : IGamePadFunctionality
{
    double vibrating = 1.0;

    public string GetGamePadName() => "Steam controller";

    public void SetVibrationPower(float power) => this.vibrating = Convert.ToDouble(Math.Clamp(power,
0.0f, 1.0f));
}

// This class is the client which receives a service.
class GamePad
{
    IGamePadFunctionality gamePadFunctionality;

    // The service is injected through the constructor and stored in the above field.
    public GamePad(IGamePadFunctionality gamePadFunctionality) => this.gamePadFunctionality =
gamePadFunctionality;

    public void Showcase()
    {
        // The injected service is used.
        string gamePadName = this.gamePadFunctionality.GetGamePadName();
        string message = $"We're using the {gamePadName} right now, do you want to change the
vibrating power?";
        Console.WriteLine(message);
    }
}

class Program
{
    static void Main(string[] args)
    {
        SteamController steamController = new();

        // We could have also passed in an XboxController, PlayStationJoystick, etc.
        // The gamepad doesn't know what it's using and doesn't need to.
        GamePad gamepad = new(steamController);

        gamepad.Showcase();
    }
}

```

Go

Go does not support classes and usually dependency injection is either abstracted by a dedicated library that utilizes reflection or generics (the latter being supported since Go 1.18^[43]).^[44] A simpler example without using dependency injection libraries is illustrated by the following example of an MVC web application.

First, pass the necessary dependencies to a router and then from the router to the controllers:

```

package router

import (
    "database/sql"
    "net/http"

```

```

    "example/controllers/users"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"

    "github.com/redis/go-redis/v9"
    "github.com/rs/zerolog"
)

type RoutingHandler struct {
    // passing the values by pointer further down the call stack
    // means we won't create a new copy, saving memory
    log      *zerolog.Logger
    db       *sql.DB
    cache    *redis.Client
    router   chi.Router
}

// connection, logger and cache initialized usually in the main function
func NewRouter(
    log *zerolog.Logger,
    db *sql.DB,
    cache *redis.Client,
) (r *RoutingHandler) {
    rtr := chi.NewRouter()

    return &RoutingHandler{
        log:    log,
        db:     db,
        cache:  cache,
        router: rtr,
    }
}

func (r *RoutingHandler) SetupUsersRoutes() {
    uc := users.NewController(r.log, r.db, r.cache)

    r.router.Get("/users/:name", func(w http.ResponseWriter, r *http.Request) {
        uc.Get(w, r)
    })
}

```

Then, the private fields of the struct can be accessed in any method that is its pointer receiver, without violating encapsulation.

```

package users

import (
    "database/sql"
    "net/http"

    "example/models"

    "github.com/go-chi/chi/v5"
    "github.com/redis/go-redis/v9"
    "github.com/rs/zerolog"
)

type Controller struct {
    log      *zerolog.Logger
    storage  models.UserStorage
    cache    *redis.Client
}

func NewController(log *zerolog.Logger, db *sql.DB, cache *redis.Client) *Controller {
    return &Controller{
        log:    log,
        storage: models.NewUserStorage(db),
        cache:  cache,
    }
}

```

```

func (uc *Controller) Get(w http.ResponseWriter, r *http.Request) {
    // note that we can also wrap logging in a middleware, this is for demonstration purposes
    uc.log.Info().Msg("Getting user")

    userParam := chi.URLParam(r, "name")

    var user *models.User
    // get the user from the cache
    err := uc.cache.Get(r.Context(), userParam).Scan(&user)
    if err != nil {
        uc.log.Error().Err(err).Msg("Error getting user from cache. Retrieving from SQL storage")
    }

    user, err = uc.storage.Get(r.Context(), "johndoe")
    if err != nil {
        uc.log.Error().Err(err).Msg("Error getting user from SQL storage")
        http.Error(w, "Internal server error", http.StatusInternalServerError)
        return
    }
}

```

Finally, the database connection initialized in the main function can be used at the data access layer:

```

package models

import (
    "database/sql"
    "time"
)

type (
    UserStorage struct {
        conn *sql.DB
    }

    User struct {
        Name      string 'json:"name" db:"name,primarykey"'
        JoinedAt  time.Time 'json:"joined_at" db:"joined_at"'
        Email     string 'json:"email" db:"email"'
    }
)

func NewUserStorage(conn *sql.DB) *UserStorage {
    return &UserStorage{
        conn: conn,
    }
}

func (us *UserStorage) Get(name string) (user *User, err error) {
    // assuming 'name' is a unique key
    query := "SELECT * FROM users WHERE name = $"
    if err := us.conn.QueryRow(query, name).Scan(&user); err != nil {
        return nil, err
    }

    return user, nil
}

```

Rust

In Rust, dependency injection is typically done through generics with static dispatch.

```

trait Logger {
    fn log(&self, message: &str);
}

```

```

struct ConsoleLogger;

impl Logger for ConsoleLogger {
    fn log(&self, message: &str) {
        println!("{message}");
    }
}

struct UserService<L: Logger> {
    logger: L,
}

impl<L: Logger> UserService<L> {
    fn new(logger: L) -> Self {
        Self { logger }
    }

    fn create_user(&self, name: &str) {
        self.logger.log(&format!("Creating user: {name}"));
        // Business logic...
    }
}

fn main() {
    let logger = ConsoleLogger;
    let service = UserService::new(logger);

    service.create_user("Alice");
}

```

However, it can also be done using dynamic dispatch with trait objects:

```

trait Logger {
    fn log(&self, message: &str);
}

struct ConsoleLogger;

impl Logger for ConsoleLogger {
    fn log(&self, message: &str) {
        println!("{message}");
    }
}

struct UserService {
    logger: Box<dyn Logger>,
}

impl UserService {
    fn new(logger: Box<dyn Logger>) -> Self {
        Self { logger }
    }

    fn create_user(&self, name: &str) {
        self.logger.log(&format!("Creating user: {name}"));
    }
}

fn main() {
    let logger = Box::new(ConsoleLogger);
    let service = UserService::new(logger);

    service.create_user("Alice");
}

```

See also

- [Architecture description language](#)
- [Factory pattern](#)
- [Inversion of control](#)
- [Mock trainwreck](#)
- [Plug-in \(computing\)](#)
- [Singleton pattern](#)
- [Strategy pattern](#)
- [Service locator pattern](#)
- [Parameter \(computer programming\)](#)
- [Quaject](#)

References

1. Seemann, Mark. "Dependency Injection is Loose Coupling" (<http://blog.ploeh.dk/2010/04/07/DependencyInjectionisLooseCoupling/>). *blog.ploeh.dk*. Retrieved 2015-07-28.
2. Seeman, Mark (October 2011). *Dependency Injection in .NET*. Manning Publications. p. 4. ISBN 9781935182504.
3. Niko Schwarz, Mircea Lungu, Oscar Nierstrasz, "Seuss: Decoupling responsibilities from static methods for fine-grained configurability", *Journal of Object Technology*, volume 11, no. 1 (April 2012), pp. 3:1–23.
4. "HollywoodPrinciple" (<http://c2.com/cgi/wiki?HollywoodPrinciple>). *c2.com*. Retrieved 2015-07-19.
5. "The Dependency Injection design pattern – Problem, Solution, and Applicability" (<http://w3sdesign.com/?gr=u01&ugr=proble>). *w3sDesign.com*. Retrieved 2017-08-12.
6. Erez, Guy (2022-03-09). "Dependency Inversion vs. Dependency Injection" (<https://betterprogramming.pub/straightforward-simple-dependency-inversion-vs-dependency-injection-7d8c0d0ed28e>). *Medium*. Retrieved 2022-12-06.
7. Mathews, Sasha (2021-03-25). "You are Simply Injecting a Dependency, Thinking that You are Following the Dependency Inversion..." (<https://levelup.gitconnected.com/you-are-simply-injecting-a-dependency-thinking-that-you-are-following-the-dependency-inversion-32632954c208>) *Medium*. Retrieved 2022-12-06.
8. "Spring IoC Container" (<https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/beans.html>). Retrieved 2023-05-23.
9. Fowler, Martin. "Inversion of Control Containers and the Dependency Injection pattern" (<http://martinfowler.com/articles/injection.html#InversionOfControl>). *MartinFowler.com*. Retrieved 4 June 2023.
10. "Dependency Injection in NET" (<https://web.archive.org/web/20150721171646/http://philkildea.co.uk/james/books/DependencyInjection.in.NET.pdf>) (PDF). *philkildea.co.uk*. p. 4. Archived from the original (<http://philkildea.co.uk/james/books/DependencyInjection.in.NET.pdf>) (PDF) on 2015-07-21. Retrieved 2015-07-18.
11. "How to explain dependency injection to a 5-year-old?" (<https://stackoverflow.com/questions/1638919/how-to-explain-dependency-injection-to-a-5-year-old>). *stackoverflow.com*. Retrieved 2015-07-18.
12. I.T., Titanium. "James Shore: Dependency Injection Demystified" (<http://www.jamesshore.com/Blog/Dependency-Injection-Demystified.html>). *www.jamesshore.com*. Retrieved 2015-07-18.

13. "To "new" or not to "new"..." (<https://web.archive.org/web/20200513185005/http://misko.hevery.com/2008/09/30/to-new-or-not-to-new/>) Archived from the original (<http://misko.hevery.com/2008/09/30/to-new-or-not-to-new/>) on 2020-05-13. Retrieved 2015-07-18.
14. "How to write testable code" (<http://www.loosecouplings.com/2011/01/how-to-write-testable-code-overview.html>). *www.loosecouplings.com*. Retrieved 2015-07-18.
15. "Writing Clean, Testable Code" (<http://www.ethanresnick.com/blog/testableCode.html>). *www.ethanresnick.com*. Retrieved 2015-07-18.
16. Sironi, Giorgio. "When to inject: the distinction between newables and injectables - Invisible to the eye" (<http://www.giorgiosironi.com/2009/07/when-to-inject-distinction-between.html>). *www.giorgiosironi.com*. Retrieved 2015-07-18.
17. "the urban canuk, eh: On Dependency Injection and Violating Encapsulation Concerns" (<http://www.bryancook.net/2011/08/on-dependency-injection-and-violating.html>). *www.bryancook.net*. Retrieved 2015-07-18.
18. "The Dependency Injection Design Pattern" ([https://msdn.microsoft.com/en-us/library/vstudio/hh323705\(v=vs.100\).aspx](https://msdn.microsoft.com/en-us/library/vstudio/hh323705(v=vs.100).aspx)). *msdn.microsoft.com*. Retrieved 2015-07-18.
19. "The Java Community Process(SM) Program - JSRs: Java Specification Requests - detail JSR# 330" (<https://jcp.org/en/jsr/detail?id=330>). *jcp.org*. Retrieved 2015-07-18.
20. "3.1. Dependency injection — Python 3: from None to Machine Learning" (<https://web.archive.org/web/20200208005839/http://python.astrotech.io/design-patterns/structural/dependency-injection.html>). Archived from the original (<https://python.astrotech.io/design-patterns/structural/dependency-injection.html>) on 2020-02-08.
21. "How Dependency Injection (DI) Works in Spring Java Application Development - DZone Java" (<https://dzone.com/articles/how-dependency-injection-di-works-in-spring-java-a>).
22. "Dependency injection and inversion of control in Python — Dependency Injector 4.36.2 documentation" (http://python-dependency-injector.ets-labs.org/introduction/di_in_python.html).
23. "How to Refactor for Dependency Injection, Part 3: Larger Applications" (<https://visualstudiomagazine.com/articles/2014/07/01/larger-applications.aspx>).
24. "A quick intro to Dependency Injection: What it is, and when to use it" (<https://www.freecodecamp.org/news/a-quick-intro-to-dependency-injection-what-it-is-and-when-to-use-it-7578c84fa88f/>). 18 October 2018.
25. "Dependency Injection |Professionalqa.com" (<https://www.professionalqa.com/dependency-injection>).
26. "What are the downsides to using Dependency Injection?" (<https://stackoverflow.com/questions/2407540/what-are-the-downsides-to-using-dependency-injection>). *stackoverflow.com*. Retrieved 2015-07-18.
27. "Dependency Injection Inversion – Clean Coder" (<https://sites.google.com/site/unclebobconsultingllc/blogs-by-robert-martin/dependency-injection-inversion>). *sites.google.com*. Retrieved 2015-07-18.
28. "Decoupling Your Application From Your Dependency Injection Framework" (<http://www.infoq.com/news/2010/01/dependency-injection-inversion>). *InfoQ*. Retrieved 2015-07-18.
29. Martin Fowler (2004-01-23). "Inversion of Control Containers and the Dependency Injection pattern – Forms of Dependency Injection" (<http://www.martinfowler.com/articles/injection.html#FormsOfDependencyInjection>). *Martinfowler.com*. Retrieved 2014-03-22.
30. "AccessibleObject (Java Platform SE 7)" (<http://docs.oracle.com/javase/7/docs/api/java/lang/reflect/AccessibleObject.html>). *docs.oracle.com*. Retrieved 2015-07-18.
31. Microsoft Learn (11 March 2026). "Dependency injection guidelines" (<https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/guidelines>). *learn.microsoft.com*. Microsoft Learn.

32. Riehle, Dirk (2000), *Framework Design: A Role Modeling Approach* (<http://www.riehle.org/computer-science/research/dissertation/diss-a4.pdf>) (PDF), Swiss Federal Institute of Technology
33. "Dependency Injection != using a DI container" (<http://www.loosecouplings.com/2011/01/dependency-injection-using-di-container.html>). *www.loosecouplings.com*. Retrieved 2015-07-18.
34. "Black Sheep » DIY-DI » Print" (<https://web.archive.org/web/20150627215638/http://blacksheep.parry.org/archives/diy-di/print>). *blacksheep.parry.org*. Archived from the original (<https://blacksheep.parry.org/archives/diy-di/print/>) on 2015-06-27. Retrieved 2015-07-18.
35. Jakarta EE (16 October 2021). "Package jakarta.inject" (<https://jakarta.ee/specifications/dependency-injection/2.0/apidocs/jakarta/inject/package-summary>). *jakarta.ee*. Jakarta EE.
36. Microsoft Learn. "Microsoft.Extensions.DependencyInjection Namespace" (<https://learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.dependencyinjection?view=net-11.0-pp>). *learn.microsoft.com*. Microsoft Learn. Retrieved 6 July 2026.
37. "Spring Tips: A POJO with annotations is not Plain" (<https://web.archive.org/web/20150715045353/http://springtips.blogspot.com/2007/07/pojo-with-annotations-is-not-plain.html>). Archived from the original (<http://springtips.blogspot.com/2007/07/pojo-with-annotations-is-not-plain.html>) on 2015-07-15. Retrieved 2015-07-18.
38. "Annotations in POJO – a boon or a curse? | Techtracer" (<http://techtracer.com/2007/04/07/annotations-in-pojo-a-boon-or-a-curse/>). 2007-04-07. Retrieved 2015-07-18.
39. *Pro Spring Dynamic Modules for OSGi Service Platforms* (<https://books.google.com/books?id=FCVnsq1ZUI0C&q=spring+pojo+annotation+free&pg=PA64>). APress. 2009-02-17. ISBN 9781430216124. Retrieved 2015-07-06.
40. Kris Jusiak (2 April 2025). "[Boost::ext].DI" (<https://boost-ext.github.io/di/>). *boost-ext.github.io*. Boost C++ Libraries.
41. Adam Lach, Walter Genovese (15 March 2025). "Function Parameter Reflection in Reflection for C++26" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3096r7.pdf>) (PDF). *open-std.org*. WG 21.
42. Wyatt Childers, Dan Katz, Barry Rezvin, Daveed Vandevoorde (19 June 2025). "Annotations for Reflection" (<https://isocpp.org/files/papers/P3394R4.html>). *isocpp.org*. WG21.
43. "Go 1.18 Release Notes - The Go Programming Language" (<https://go.dev/doc/go1.18>). *go.dev*. Retrieved 2024-04-17.
44. "Awesome Go – dependency injection" (<https://github.com/avelino/awesome-go?tab=readme-ov-file#dependency-injection>). *Github*. April 17, 2024. Retrieved April 17, 2024.

External links

- [Composition Root](http://blog.ploeh.dk/2011/07/28/CompositionRoot/) by Mark Seemann (<http://blog.ploeh.dk/2011/07/28/CompositionRoot/>)
- [A beginners guide to Dependency Injection](https://www.theserverside.com/news/1321158/A-beginners-guide-to-Dependency-Injection) (<https://www.theserverside.com/news/1321158/A-beginners-guide-to-Dependency-Injection>)
- [Dependency Injection & Testable Objects: Designing loosely coupled and testable objects](http://www.ddj.com/185300375) (<http://www.ddj.com/185300375>) - Jeremy Weiskotten; Dr. Dobb's Journal, May 2006.
- [Design Patterns: Dependency Injection -- MSDN Magazine, September 2005](http://www.griffincaprio.com/blog/2018/04/design-patterns-dependency-injection.html) (<http://www.griffincaprio.com/blog/2018/04/design-patterns-dependency-injection.html>)
- [Martin Fowler's original article that introduced the term Dependency Injection](http://martinfowler.com/articles/injection.html) (<http://martinfowler.com/articles/injection.html>)
- [P of EAA: Plugin](http://martinfoowler.com/eaCatalog/plugin.html) (<http://martinfoowler.com/eaCatalog/plugin.html>)
- [The Rich Engineering Heritage Behind Dependency Injection](https://web.archive.org/web/20080313170630/http://www.javalobby.org/articles/di-heritage/) (<https://web.archive.org/web/20080313170630/http://www.javalobby.org/articles/di-heritage/>) - Andrew McVeigh - A detailed history of dependency injection.

- [What is Dependency Injection? \(http://tutorials.jenkov.com/dependency-injection/index.html\)](http://tutorials.jenkov.com/dependency-injection/index.html)
- An alternative explanation - Jakob Jenkov
- [Writing More Testable Code with Dependency Injection -- Developer.com, October 2006 \(http://www.developer.com/net/net/article.php/3636501\)](http://www.developer.com/net/net/article.php/3636501) Archived (<https://web.archive.org/web/20080311121626/http://www.developer.com/net/net/article.php/3636501>) 2008-03-11 at the Wayback Machine
- [Managed Extensibility Framework Overview -- MSDN \(http://msdn.microsoft.com/en-us/library/dd460648.aspx\)](http://msdn.microsoft.com/en-us/library/dd460648.aspx)
- [Old fashioned description of the Dependency Mechanism by Hunt 1998 \(https://web.archive.org/web/20120425150101/http://www.midmarsh.co.uk/planetjava/tutorials/language/WatchingtheObservables.PDF\)](https://web.archive.org/web/20120425150101/http://www.midmarsh.co.uk/planetjava/tutorials/language/WatchingtheObservables.PDF)
- [Refactor Your Way to a Dependency Injection Container \(http://blog.thecodewhisperer.com/2011/12/07/refactor-your-way-to-a-dependency-injection-container/\)](http://blog.thecodewhisperer.com/2011/12/07/refactor-your-way-to-a-dependency-injection-container/)
- [Understanding DI in PHP \(http://php-di.org/doc/understanding-di.html\)](http://php-di.org/doc/understanding-di.html)
- [You Don't Need a Dependency Injection Container \(https://medium.com/@wrong.about/you-dont-need-a-dependency-injection-container-10a5d4a5f878\)](https://medium.com/@wrong.about/you-dont-need-a-dependency-injection-container-10a5d4a5f878)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Dependency_injection&oldid=1364559000"